

# Production Design Report for the Bare Systems Edge CLI

## Executive summary

The most robust path for a production-ready customer-facing CLI is a **single statically linked bare-systems binary** that acts as an installer, orchestrator, and support tool for **Docker Compose-based edge deployments**. The CLI should not replace Docker; it should **standardize and automate** how Bare Systems composes, validates, enrolls, runs, updates, and diagnoses a deployment on customer hardware. That design best matches Docker Compose's strengths as a multi-container application runtime and keeps upgrades centered on **image updates** rather than host-library drift. Docker Compose is expressly designed for defining and running multi-container applications, supports composition from multiple files, modular **include**, profiles, secrets, healthchecks, and machine-readable outputs that are well-suited to a customer support CLI. <sup>1</sup>

For Bare Systems specifically, the CLI should treat a deployment as: **a project manifest ( `edge.yml` ), environment/secrets material ( `.env` , **secret files** ), one or more resolved Compose templates, and a runtime state directory**. The core deployment should install only the minimum essential services by default: **Tardigrade reverse proxy, Bear Claw web UI, and Bear Claw AI agent runtime integration**. Optional modules such as Koala, Polar, Kodiak, and URSA should be activated through **profiles and manifests**, rather than by hard-coding service combinations. That approach directly aligns with Docker Compose profiles, which only start services assigned to a profile when the profile is activated; unassigned services always start. <sup>2</sup>

The CLI's packaging strategy should be **phased**. For first GA, ship: **GitHub Releases + install script + Homebrew tap + Linux `.deb` / `.rpm` packages**. These channels cover the likely operational surface area quickly without forcing the complexity and security compromises of Snap classic confinement or the lower-priority Windows runtime story. Homebrew taps are Git-backed third-party repositories and will increasingly require explicit trust for non-official taps; this makes a dedicated `Bare-Systems/homebrew-tap` viable, but it must be documented clearly as a trust decision. Debian repositories should use normal APT trust chains with signed repository metadata and avoid deprecated `apt-key`; RPM packages should be signed with `rpm-sign`. Winget is useful for developer/operator workstations, but because the edge runtime itself is Linux- and systemd-centric, Winget should be treated as a secondary distribution channel for the CLI rather than a primary deployment target. <sup>3</sup>

Security-wise, the main architectural hazard is not the CLI binary itself, but the surrounding deployment model. Docker's daemon has a significant attack surface and ordinarily runs with root privileges unless rootless mode is used. The Docker documentation explicitly warns about the daemon attack surface, and operationally, access to the Docker socket or privileged Docker operations must be treated as high-trust. The CLI should therefore keep secrets out of logs, avoid shelling with interpolated secrets, write config and secret files with restrictive permissions, avoid broad container privileges, prefer running containers as non-root where supported, and use Compose secrets where possible. Compose supports per-service secret

grants and mounts them read-only under `/run/secrets`, which is a much better fit than environment variables for durable secrets. <sup>4</sup>

The CI/CD model should use **GitHub Actions** and a **matrix-driven build/test pipeline**. GitHub Actions supports matrix strategies natively, artifact retention controls, and artifact attestations that provide cryptographically signed build provenance. GitHub-hosted macOS arm64 runners exist, but GitHub's own docs note arm64 macOS limitations and also note that community actions may not all be compatible. For Linux ARM64 and more exhaustive platform testing, self-hosted runners are appropriate, and GitHub documents support for Linux/macOS/Windows ARM64 self-hosted runners. Releases should publish signed checksums, provenance attestations, and package artifacts together. This is compatible with Go's reproducible-build tooling: Go documents `-trimpath` as improving build reproducibility, Go modules as enabling reproducible dependency graphs, and the Go project's own reproducible-build work strongly supports a "trust but verify" release posture. <sup>5</sup>

The recommended implementation language is **Go**, because it produces a single binary cleanly across Linux and macOS targets, fits GitHub Actions well, and aligns with the desired deployment and packaging shape. The CLI should emit consistent **human-readable and JSON output**, have explicit **idempotency rules**, and provide a first-class `doctor` and **diagnostics bundle** workflow. Because Docker Compose already supports JSON-formatted outputs such as `docker compose ps --format json`, `docker inspect` JSON output, and canonical config rendering via `docker compose config`, the CLI can collect structured runtime evidence without inventing its own container-state model. <sup>6</sup>

Unspecified items remain, and they matter: **Portal API routes and auth flows, exact image names/tags/digests, exact set of mandatory modules, whether rootless Docker must be supported, and the exact telemetry contract** are all still undefined. This report therefore proposes a production architecture that isolates those unknowns behind explicit config schemas and interfaces rather than hard-coding assumptions.

## Recommended architecture and operating model

The deployment CLI should own exactly four concerns: **host preparation, config resolution, Compose lifecycle management, and Portal enrollment/reporting**. It should **not** own long-running container orchestration logic beyond starting and supervising a Compose application; Docker Compose and systemd already cover those responsibilities well. Docker Compose can merge files, modularize with `include`, inherit reusable blocks with YAML fragments and `extends`, resolve interpolation from environment sources, and output a fully resolved model with `docker compose config`. That means the CLI can function as a high-level product layer over standard Docker primitives rather than a parallel orchestrator. <sup>7</sup>

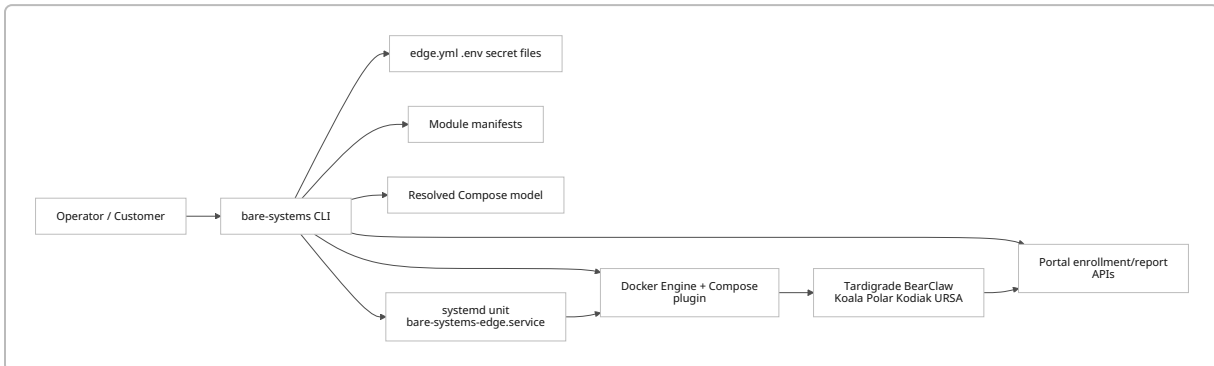
The core product model should be:

- **Base system:** the `bare-systems` binary, installed on the host.
- **Project directory:** an on-disk deployment root, for example `/opt/bare-systems`.
- **Deployment config:** `edge.yml` plus `.env` plus per-secret files.
- **Module manifests:** versioned, local YAML describing module-to-profile/service mappings.

- **Generated artifacts:** resolved Compose YAML, lock files, health state, last report, diagnostics bundles.
- **Runtime:** Docker Compose project and a systemd unit that starts/stops it.

This architecture keeps the CLI single-binary while still allowing the runtime to stay containerized, which matches the user’s stated preference for Docker image-centric upgrades and lower host library churn.

## Component relationship diagram



## Recommended on-disk layout

A Linux-first layout should prefer conventional, supportable paths:

```

/usr/bin/bare-systems           # CLI binary
/etc/bare-systems/
  edge.yml                     # deployment configuration
  .env                         # non-secret interpolation values
  secrets/
    portal-token               # one file per secret, 0600
    tls-key
    tls-cert
/opt/bare-systems/
  compose/
    base.compose.yml
    generated.compose.yml
    lock.compose.yml
  manifests/
    core.yml
    koala.yml
    polar.yml
    kodiak.yml
    ursa.yml
  state/
    install.json
    enroll.json
    report-cache.json

```

```
health.json
doctor/
bundles/
  diagnostics-YYYYMMDD-HHMMSS.tar.gz
/var/log/bare-systems/
cli.log
```

The split between `/etc` and `/opt` is intentional. `/etc` holds operator-managed config and secrets; `/opt` holds product-controlled templates, manifests, and generated runtime state. This reduces accidental overwrite risk on upgrades and makes support workflows clearer.

## systemd integration model

A systemd unit should supervise the Compose project rather than each individual container. systemd service units are the normal Linux mechanism for supervising processes, and `systemctl` provides standard lifecycle operations. `systemd.exec` also exposes hardening knobs such as protected filesystems, private temporary directories, and credentials features, though exact support varies by distro/version. <sup>8</sup>

Recommended design:

- `bare-systems init` installs the systemd unit and any tmpfiles config.
- `bare-systems enable` enables and starts the service.
- systemd runs `bare-systems service run --project /opt/bare-systems`.
- That subcommand executes the resolved Compose lifecycle:
- `docker compose up -d`
- monitor/report health
- on stop, `docker compose down` only when explicitly requested
- The CLI should **not** rely on systemd to supervise each container; container restart behavior should remain in Docker/Compose.

A production-grade unit should include at least:

- `After=docker.service network-online.target`
- `Requires=docker.service`
- `Restart=on-failure`
- `WorkingDirectory=/opt/bare-systems/compose`
- `ExecStart=/usr/bin/bare-systems service run`
- `ExecReload=/usr/bin/bare-systems reload`
- `User=root` for install/start only if necessary, because Docker access is privileged anyway
- hardening options where compatible, but without breaking Docker socket access

Because Docker access is high-trust, there is limited value in over-hardening the CLI service while leaving unrestricted Docker access available; the important control is who can invoke the CLI and who can access Docker.

## Configuration schema, module manifests, and Compose resolution

A good config model should separate **operator intent** from **runtime realization**. Operators edit `edge.yml`; the CLI resolves that into a canonical Compose model plus a deterministic lock artifact.

### Proposed `edge.yml` schema

This is a recommended product schema, not an official standard.

```
apiVersion: bare.systems/v1alpha1
kind: EdgeDeployment

metadata:
  name: site-001
  customer: acme-industrial
  description: Main warehouse edge node

spec:
  channel: stable                # canary | beta | stable
  projectName: bare-systems
  customerId: "<unspecified>"
  siteId: "<unspecified>"
  enroll:
    enabled: true
    portalBaseUrl: "https://<unspecified>"
    tokenFile: "/etc/bare-systems/secrets/portal-token"
    labels:
      region: us-east
      role: warehouse
  runtime:
    composeProjectDirectory: "/opt/bare-systems/compose"
    dockerContext: "default"
    profiles:
      - core
      - ai
    imagePolicy:
      mode: tag                  # tag | digest
      allowFloatingTags: false
    restartPolicy: unless-stopped
  modules:
    core:
      enabled: true
      ui: true
      reverseProxy: true
    ai:
      enabled: true
```

```

    provider: "local-openai-compatible"
    endpoint: "http://host.docker.internal:11434/v1"
    model: "unspecified"
  koala:
    enabled: false
  polar:
    enabled: false
  kodiak:
    enabled: false
  ursa:
    enabled: false
  networking:
    publicHttpPort: 80
    publicHttpsPort: 443
    adminBindAddress: "0.0.0.0"
  storage:
    root: "/opt/bare-systems"
    backups: "/opt/bare-systems/backups"
  reporting:
    heartbeatInterval: "5m"
    includeStats: true
    includeContainerHealth: true
  support:
    diagnosticsRetention: 10
    redactSecrets: true

```

## Proposed `.env` contract

Use `.env` for **non-secret interpolation only**. Docker Compose supports interpolation from the shell environment, `.env` in the working directory when `--env-file` is not set, and `--env-file` or project `.env` sources, with precedence rules. Values in Compose files support Bash-like `${VARIABLE}` syntax.

9

Recommended `.env` examples:

```

BARE_CHANNEL=stable
BARE_PROJECT_NAME=bare-systems
PUBLIC_HTTP_PORT=80
PUBLIC_HTTPS_PORT=443
ADMIN_BIND_ADDRESS=0.0.0.0
AI_PROVIDER=local-openai-compatible
AI_ENDPOINT=http://host.docker.internal:11434/v1
AI_MODEL=unspecified

```

Do **not** put enrollment tokens, API keys, or private keys into `.env` unless there is no alternative. When secrets are needed inside containers, mount them as Compose secrets instead.

## Proposed module manifest schema

Each module should be defined declaratively so the CLI can build deployments without embedding every service definition in code.

```
apiVersion: bare.systems/v1alpha1
kind: ModuleManifest

metadata:
  name: koala
  version: 1
  description: Camera and home security services

module:
  id: koala
  defaultEnabled: false
  profiles:
    - koala

images:
  koala-orchestrator:
    image: "registry.example.com/bare/koala-orchestrator:unspecified"
  koala-worker:
    image: "registry.example.com/bare/koala-worker:unspecified"

services:
  koala-orchestrator:
    composeService: koala-orchestrator
    enabledWhen:
      module: koala
    dependsOn: []
    health:
      type: http
      url: "http://localhost:8081/health"
  koala-worker:
    composeService: koala-worker
    enabledWhen:
      module: koala
    dependsOn:
      - koala-orchestrator
    health:
      type: exec
      command: ["CMD", "/app/healthcheck"]

config:
  required:
    - KOALA_SITE_ID
```

```

optional:
  - KOALA_CAMERA_RETENTION_DAYS

ports: []
volumes:
  - koala-data
secrets: []

```

## Compose template patterns

The recommended pattern is:

- one **base Compose file** for always-on platform concerns
- one or more **module include files**
- optional **override** files for site-specific or support-time changes
- YAML **fragments/anchors** for reusable service snippets
- **profiles** for conditional service activation
- optional **digest lock override** generated by the CLI for release pinning

Docker Compose supports YAML fragments/anchors, recursive `include`, merges with `-f`, and profiles. `docker compose config` can render the actual data model, output YAML or JSON, and even generate an override file with image digests using `--lock-image-digests`; it can also resolve image tags to digests with `--resolve-image-digests`. <sup>10</sup>

Recommended file set:

```

base.compose.yml
modules/
  core.compose.yml
  koala.compose.yml
  polar.compose.yml
  kodiak.compose.yml
  ursa.compose.yml
site.override.compose.yml
lock.compose.yml

```

Example pattern:

```

# base.compose.yml
services:
  tardigrade:
    image: ${TARDIGRADE_IMAGE}
    profiles: ["core"]
  bearclaw-web:
    image: ${BEARCLAW_WEB_IMAGE}

```



```
profiles: ["core"]
bearclaw-agent:
  image: ${BEARCLAW_AGENT_IMAGE}
  profiles: ["ai"]
```

Module includes should be **non-conflicting**, because `include` copies resource definitions into the current model and warns on conflicts instead of merging them. That makes `include` good for clean module boundaries, while `-f` overrides remain the right tool for environment-specific modifications. <sup>11</sup>

## Profile resolution rules

Recommended resolution algorithm:

1. Always enable `core`.
2. Enable `ai` if Bear Claw AI is configured.
3. For each module with `enabled: true`, add its declared profile(s).
4. Reject unknown module/profile combinations.
5. Render the canonical model with:
6. base
7. module includes
8. optional site override
9. optional lock/digest override
10. Validate with `docker compose config -q`.
11. Save the rendered canonical model.

This yields deterministic, supportable behavior and maps cleanly to Compose semantics.

## CLI UX, idempotency, diagnostics, and health model

The CLI should feel like an installer and support tool, not like a generic DevOps framework. That means explicit verbs, stable JSON output, well-defined error codes, and predictable idempotency.

### Command surface

Recommended command set:

```
bare-systems version
bare-systems init
bare-systems validate
bare-systems enroll
bare-systems install
bare-systems start
bare-systems stop
```

```
bare-systems restart
bare-systems status
bare-systems ps
bare-systems logs
bare-systems update
bare-systems rollback
bare-systems doctor
bare-systems bundle
bare-systems report
bare-systems config render
bare-systems config diff
bare-systems module list
bare-systems module enable <name>
bare-systems module disable <name>
bare-systems service install
bare-systems service uninstall
```

Recommended semantics:

- `init` : creates directories, writes default config if absent, validates host prerequisites.
- `validate` : validates config, module manifests, secrets references, Compose renderability.
- `enroll` : exchanges a one-time Portal token for a device enrollment record and a renewable credential.
- `install` : pulls images, writes generated Compose files, installs systemd unit, optionally enrolls.
- `start` / `stop` / `restart` : lifecycle commands.
- `status` : high-level health summary.
- `ps` : machine-readable service/container list.
- `update` : refreshes images/config within policy.
- `rollback` : reverts to prior known-good artifact set.
- `doctor` : diagnostics with actionable remediation.
- `bundle` : support artifact tarball.
- `report` : send heartbeat/status to Portal.
- `config render` : output canonical Compose YAML/JSON.
- `config diff` : compare current desired state vs running state.

## Flags and output conventions

Every substantive command should support:

- `--project-dir`
- `--config`
- `--env-file`
- `--profile`
- `--json`
- `--quiet`
- `--yes`

- `--portal-url`
- `--token-file`
- `--timeout`

JSON mode should output a stable envelope:

```
{
  "ok": true,
  "command": "status",
  "code": "OK",
  "message": "Deployment healthy",
  "data": { ... },
  "warnings": [],
  "errors": [],
  "timestamp": "2026-06-13T12:34:56Z"
}
```

This JSON contract is particularly important because Docker already exposes structured state through `docker compose ps --format json`, `docker compose stats --format json`, and `docker inspect` JSON output, making it straightforward to map container/runtime data into a stable product API.

12

## Error codes

Recommended normalized exit code model:

| Exit code | Symbol       | Meaning                                |
|-----------|--------------|----------------------------------------|
| 0         | OK           | Success                                |
| 1         | ERR_GENERIC  | Unclassified failure                   |
| 2         | ERR_USAGE    | Invalid CLI arguments                  |
| 3         | ERR_CONFIG   | Invalid config/schema/value            |
| 4         | ERR_PREREQ   | Missing Docker/system requirement      |
| 5         | ERR_AUTH     | Enrollment/auth failure                |
| 6         | ERR_NETWORK  | Network or Portal connectivity failure |
| 7         | ERR_RUNTIME  | Compose/runtime failure                |
| 8         | ERR_HEALTH   | Services started but unhealthy         |
| 9         | ERR_UPDATE   | Update failed                          |
| 10        | ERR_ROLLBACK | Rollback failed                        |

| Exit code | Symbol                       | Meaning                         |
|-----------|------------------------------|---------------------------------|
| 11        | <code>ERR_DIAGNOSTICS</code> | Bundle/doctor failed            |
| 12        | <code>ERR_PERMISSIONS</code> | Filesystem or privilege failure |

The CLI should also emit symbolic codes in JSON to decouple clients from numeric exit codes.

## Idempotency rules

Idempotency is critical for customer support and automation. Recommended rules:

- `init` is idempotent: create missing directories/files, never overwrite without `--force`.
- `install` is idempotent: re-running should reconcile to desired state and not duplicate side effects.
- `enroll` is idempotent: if already enrolled, return current enrollment unless `--re-enroll`.
- `service install` is idempotent: re-running updates the unit if changed.
- `start` when already running returns success with `"alreadyRunning": true`.
- `stop` when already stopped returns success with `"alreadyStopped": true`.
- `update` without newer allowed artifacts returns success with `"updated": false`.
- `rollback` requires a known prior revision; otherwise clear `ERR_ROLLBACK`.

## Health model

Use a layered health model:

- **Level 1:** host prerequisites OK
- **Level 2:** Compose project valid and containers created
- **Level 3:** containers running
- **Level 4:** container health checks passing
- **Level 5:** product health checks passing
- **Level 6:** Portal reporting healthy

Compose supports service `healthcheck` overrides, and `depends_on` long syntax can require `service_healthy` before starting dependent services. This is the right foundation for product health orchestration. <sup>13</sup>

Recommended health sources:

- container state from `docker compose ps --format json`
- health state from container health checks
- product checks such as:
  - Tardigrade proxy responds on localhost
  - Bear Claw web returns HTTP 200
  - AI provider endpoint reachable
  - optional module endpoints reachable
  - report delivery success and staleness

## Diagnostics bundle format

Recommended format: `tar.gz` with manifest + structured JSON + selected text logs.

```
bundle/  
  manifest.json  
  version.json  
  system/  
    uname.txt  
    os-release.txt  
    docker-version.txt  
    docker-info.txt  
    systemd-unit.txt  
    journal.txt  
  config/  
    edge.redacted.yml  
    env.redacted  
    compose.rendered.yml  
    compose.lock.yml  
    module-manifests/  
  runtime/  
    compose-ps.json  
    inspect/  
      <container>.json  
    stats.json  
    logs/  
      <service>.log  
  health/  
    doctor.json  
    checks.json  
  portal/  
    enroll.redacted.json  
    last-report.redacted.json
```

Bundle rules:

- redact secrets and tokens
- include hashes of redacted values when useful for correlation
- always include a `manifest.json` with capture time, CLI version, config revision, platform, and module/profile set
- generate with a size budget and log truncation policy

# Security, CI/CD, packaging, and release engineering

## Security requirements

The CLI will often run where Docker is available. Docker explicitly documents the daemon attack surface and states that the daemon ordinarily requires root privileges unless rootless mode is used. That means the **security boundary to care about first is privileged Docker access.** <sup>14</sup>

Recommended security requirements:

### 1. Secrets handling

2. Store long-lived secrets in files under `/etc/bare-systems/secrets`, mode `0600`, root-owned.
3. Never log secrets, never echo token values, never include secret contents in diagnostics.
4. Prefer Compose secrets over environment variables for container-facing secrets because Compose mounts them read-only under `/run/secrets/<name>`. <sup>15</sup>

### 5. Enrollment token handling

6. Treat the initial Portal token as one-time or short-lived input.
7. Immediately exchange it for a renewable device credential.
8. Delete or archive the bootstrap token after successful use unless policy requires retention.
9. Never persist raw bootstrap tokens in plaintext runtime state if avoidable.

### 10. Least privilege

11. Containers should run as non-root where images support it; in Compose, `user` overrides the container process user, and if it is not set the default is the image's `USER` or root. <sup>16</sup>
12. Prefer `read_only: true` for applicable containers. Compose supports read-only service filesystems. <sup>17</sup>
13. Use `security_opt: [no-new-privileges]` where compatible. Compose supports this syntax. <sup>18</sup>
14. Drop unnecessary capabilities with `cap_drop`. Compose supports explicit capability dropping. <sup>19</sup>

### 15. File permissions

16. `edge.yml`: `0640` root:bare-systems or root:docker-ops equivalent
17. `.env`: `0640`
18. `secrets`: `0600`

19. generated runtime files: `0640` or tighter

20. diagnostics bundles: `0600` by default

### 21. Image integrity

22. For stable rollouts, pin images by digest in a generated lock override using Compose digest-resolution features. Docker Compose can resolve or lock image digests through `docker compose config`. <sup>20</sup>

### 23. Package and release integrity

24. Publish checksums for all assets.

25. Generate GitHub artifact attestations for build provenance. GitHub documents artifact attestations as cryptographically signed provenance and integrity information and notes they can include SBOMs. <sup>21</sup>

26. Optionally add Sigstore/Cosign blob signing for release assets; Sigstore documents `cosign sign-blob` and `verify-blob` for standard files and bundles. <sup>22</sup>

## CI/CD and release pipeline

A production pipeline should include:

### 1. PR CI

2. lint

3. unit tests

4. schema validation tests

5. Linux integration tests with Docker

6. generated CLI snapshot tests

7. doctor/bundle tests

### 8. Main branch CI

9. full integration suite

10. e2e Compose lifecycle tests

11. package build tests

12. artifact uploads

### 13. Release workflow

14. tagged semver release

15. matrix builds for binaries/packages
16. checksums
17. attestations
18. package publication
19. changelog
20. staged channel publication

GitHub Actions supports matrix strategies for systematically varying OS/arch/build variables; artifacts can be uploaded/downloaded between jobs; artifact retention is configurable; and artifact attestations provide build provenance. <sup>23</sup>

## Recommended CI matrix

| Purpose              | OS                    | Arch            | Runner recommendation        | Notes                                           |
|----------------------|-----------------------|-----------------|------------------------------|-------------------------------------------------|
| Release binary build | Linux                 | amd64           | GitHub-hosted                | standard                                        |
| Release binary build | Linux                 | arm64           | self-hosted or larger runner | safest for native validation                    |
| Release binary build | macOS                 | amd64           | GitHub-hosted Intel          | useful for Homebrew/macOS support               |
| Release binary build | macOS                 | arm64           | GitHub-hosted arm64          | watch action compatibility limits <sup>24</sup> |
| Unit tests           | Linux                 | amd64           | GitHub-hosted                | fastest feedback                                |
| Integration tests    | Linux                 | amd64           | GitHub-hosted with Docker    | main Docker/Compose path                        |
| Integration tests    | Linux                 | arm64           | self-hosted ARM64            | highest-confidence edge simulation              |
| Smoke tests          | macOS                 | amd64/<br>arm64 | GitHub-hosted                | binary/install-only, no full Linux runtime      |
| Package validation   | Debian/<br>Ubuntu     | amd64/<br>arm64 | container or VM              | <code>.deb</code> install/remove/upgrade        |
| Package validation   | RHEL/Rocky/<br>Fedora | amd64/<br>arm64 | VM/self-hosted               | <code>.rpm</code> install/remove/upgrade        |

GitHub documents GitHub-hosted runner support and ARM64 runner caveats; self-hosted runner docs explicitly support Linux/macOS/Windows ARM64. <sup>25</sup>



## Build reproducibility

Recommended Go build posture:

- `CGO_ENABLED=0` for the CLI unless a native library forces otherwise
- `-trimpath` to remove filesystem paths from binaries
- `-buildvcs=false` for reproducible public assets if you do not want repository-state stamping, or `true/auto` if you intentionally want embedded provenance and can reproduce it cleanly
- tagged module versions only
- clean `go.mod` / `go.sum`
- fixed build container or pinned runner image for release jobs

Go documents `-trimpath` as improving build reproducibility, `-buildvcs` as controlling VCS stamping, and modules as recording precise dependency requirements for reproducible builds. The Go project's own reproducible-build work is a strong primary-source basis for this recommendation. <sup>26</sup>

## Packaging options comparison

| Channel                 | Audience                      | Strengths                                          | Weaknesses                                                                                                          | Recommendation                    |
|-------------------------|-------------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| GitHub release tarballs | universal                     | simplest, immediate, works with install script     | manual upgrades without package manager                                                                             | <b>Yes, GA day one</b>            |
| Install script          | Linux/macOS                   | fast bootstrap, can verify checksum/signature      | trust and curl-pipe concerns; must be carefully documented                                                          | <b>Yes, GA day one</b>            |
| <code>.deb</code>       | Debian/Ubuntu customers       | native install/upgrade/remove, systemd integration | repo signing and QA overhead                                                                                        | <b>Yes, GA day one</b>            |
| <code>.rpm</code>       | RHEL/Rocky/Fedora/SUSE family | native enterprise packaging                        | distro variability, signing + repo overhead                                                                         | <b>Yes, GA day one</b>            |
| Homebrew tap            | macOS/Linux operators         | familiar install UX, update path                   | third-party tap trust, Ruby formula maintenance                                                                     | <b>Yes, GA day one</b>            |
| Snap                    | Linux desktop/server          | transactional updates and store distribution       | likely requires classic confinement for Docker/system-level access; classic is riskier and requires manual approval | <b>Defer unless strong demand</b> |

| Channel | Audience          | Strengths                                  | Weaknesses                                                                       | Recommendation           |
|---------|-------------------|--------------------------------------------|----------------------------------------------------------------------------------|--------------------------|
| Winget  | Windows operators | distribution for dev/operator workstations | runtime itself is Linux/systemd-centric; submission/validation workflow overhead | <b>Secondary channel</b> |

Supporting evidence:

- Homebrew taps are third-party Git repositories and non-official taps are executable code that can run with the user's privileges; Homebrew is moving toward explicit trust for them. <sup>27</sup>
- APT signed repositories rely on repository metadata trust chains; `apt-key` is deprecated and should be avoided. <sup>28</sup>
- RPM packages can be signed with `rpmsign`. <sup>29</sup>
- WinGet manifests are YAML metadata used to install and upgrade software, and submissions are automatically validated and tested. <sup>30</sup>
- Snap classic confinement is equivalent to traditional full system access, is inherently riskier, and requires manual approval and `--classic` installation. <sup>31</sup>

## Packaging implementation guidance

### apt/deb

Use a normal `.deb` package that installs:

- `/usr/bin/bare-systems`
- `/usr/lib/systemd/system/bare-systems-edge.service`
- `/etc/bare-systems/edge.yml.example`
- tmpfiles or postinst setup if required

Do not rely on `apt-key`; Debian documents it as deprecated. Signed repository metadata should be used for any hosted apt repository. <sup>32</sup>

### rpm

Build an RPM package with a spec installing the same file set. RPM's spec format supports conditionals for architecture/OS-specific handling. Sign the resulting RPMs with `rpmsign`. <sup>33</sup>

### Homebrew

Publish a dedicated tap such as `Bare-Systems/homebrew-tap`. Homebrew documents taps as external Git repositories, recommends the `homebrew-` naming convention, and makes trust an explicit security concern for non-official taps. <sup>34</sup>

## Snap

If pursued later, assume the CLI will likely need `classic` confinement because it must interact with Docker and system-level resources. Canonical states classic confinement is the most permissive model and is riskier because it lacks normal sandboxing protections. <sup>31</sup>

## Winget

Publish as a Windows operator convenience channel only. Microsoft documents manifest creation, validation, structured repository submission, and testing in sandbox/automation. <sup>30</sup>

# Development epics, testing strategy, documentation, and rollout plan

## Epics and atomic issues

The program should be split into a small number of epics with well-bounded, Codex-friendly issues.

## Epic table and effort estimates

| Epic                     | Scope                                                                          | Effort |
|--------------------------|--------------------------------------------------------------------------------|--------|
| CLI foundation           | command framework, config loading, logging, JSON output, error codes           | Medium |
| Deployment model         | <code>edge.yml</code> , manifest schema, compose rendering, profile resolution | Large  |
| Host/runtime integration | Docker checks, systemd install, lifecycle commands                             | Medium |
| Enrollment and reporting | Portal auth, report contract, retries, local cache                             | Medium |
| Health and doctor        | health checks, diagnostics, bundle generation                                  | Medium |
| Packaging and release    | binaries, checksums, attestations, deb/rpm/homebrew                            | Large  |
| Test infrastructure      | unit/integration/e2e/upgrade matrix and mocks                                  | Large  |
| Docs and support         | user docs, admin docs, runbooks, troubleshooting                               | Medium |

## Detailed epic breakdown

### CLI foundation

Atomic issues:

### Issue: bootstrap Go CLI skeleton

Acceptance criteria:

- Binary name is `bare-systems`.
- Supports `version`, `help`, and global flags.
- Global `--json`, `--quiet`, `--config`, `--project-dir` work consistently.

### Issue: standardized output and exit codes

Acceptance criteria:

- All commands emit consistent JSON envelope with `ok`, `code`, `message`, `data`.
- Exit codes map to documented symbolic codes.
- Human-readable mode remains concise and actionable.

### Issue: structured logging and redaction

Acceptance criteria:

- Logs have levels and correlation IDs.
- Secret-like values are redacted.
- `--verbose` and support logs are supported.

### Deployment model

#### Issue: define `edge.yml` schema and validation

Acceptance criteria:

- Schema documented and versioned.
- Validation catches unknown keys if strict mode enabled.
- Helpful diagnostics identify path and remediation.

#### Issue: define module manifest schema

Acceptance criteria:

- Module enablement maps to profiles and services.
- Image refs configurable and overrideable.
- Schema versioning in place.

#### Issue: implement Compose renderer

Acceptance criteria:

- Resolves base + module includes + overrides.
- Produces canonical Compose YAML.
- Validates via `docker compose config -q`.
- Saves rendered model and metadata.

**Issue: implement image lock generation**

Acceptance criteria:

- Can generate digest override file.
- Lock file references exact digests for stable deployments.
- Update logic can compare lock revisions.

**Host/runtime integration****Issue: host prerequisite checks**

Acceptance criteria:

- Detect Docker Engine, Compose plugin, disk/memory thresholds, systemd availability.
- Emits machine-readable failures with remediation.

**Issue: systemd service installer**

Acceptance criteria:

- Installs unit file and enables/disables it.
- Safe re-run behavior.
- Supports uninstall.

**Issue: lifecycle commands**

Acceptance criteria:

- `start`, `stop`, `restart`, `status`, `ps`, `logs` implemented.
- `status` uses structured compose/docker data.
- Idempotency behavior documented and tested.

**Enrollment and reporting****Issue: Portal client abstraction**

Acceptance criteria:

- Interface hides exact endpoint details.
- Supports enroll, heartbeat/report, fetch policy if applicable.
- Retries/backoff/timeouts implemented.

**Issue: secure enrollment flow**

Acceptance criteria:

- Reads bootstrap token from file/stdin/env.
- Exchanges for device credential.
- Never prints token value.
- Stores resulting credential under restrictive permissions.

**Issue: heartbeat/report pipeline**

Acceptance criteria:

- Sends deployment version, active profiles/modules, health summary, optional stats.
- Queues failed reports locally and retries.

**Health and doctor****Issue: health status engine**

Acceptance criteria:

- Aggregates container state, health checks, and product endpoint checks.
- Distinguishes degraded vs failed vs unknown.

**Issue: `doctor` diagnostics**

Acceptance criteria:

- Checks Docker reachability, compose validity, ports, disk space, time drift, portal connectivity, image status.
- Provides remediation guidance.

**Issue: diagnostics bundle**

Acceptance criteria:

- Generates tar.gz with manifest and redacted artifacts.
- Includes runtime JSON outputs and selected logs.
- Size bounded and tested.

**Packaging and release****Issue: GitHub Actions build matrix**

Acceptance criteria:

- Builds linux/darwin amd64/arm64 binaries.
- Uploads artifacts and checksums.
- Produces draft or final release.

**Issue: provenance and signing**

Acceptance criteria:

- Artifact attestations enabled.
- Signed checksum file published.
- Verification docs included.

**Issue: `.deb` and `.rpm` packaging**

Acceptance criteria:

- Native install/remove/upgrade tested.

- Installs binary, examples, systemd unit.
- Signed artifacts produced.

#### **Issue: Homebrew tap automation**

Acceptance criteria:

- Formula template published and updated on release.
- Install instructions validated on Apple Silicon and Intel macOS.

#### **Issue: install script**

Acceptance criteria:

- Detects OS/arch.
- Downloads correct asset.
- Verifies checksum/signature.
- Supports non-interactive flags.

#### **Test infrastructure**

##### **Issue: unit test baseline**

Acceptance criteria:

- Schema, parsers, error mapping, redaction, report payload generation covered.

##### **Issue: integration suite with Docker**

Acceptance criteria:

- Runs local mock Portal + disposable Compose project.
- Tests install, start, stop, status, doctor, bundle.

##### **Issue: e2e upgrade/rollback suite**

Acceptance criteria:

- Exercises versions N-1 → N upgrade and rollback to N-1.
- Preserves config and secrets.

##### **Issue: Portal mocks/test data**

Acceptance criteria:

- Mock server supports enroll/report failure modes, latency, auth errors, stale policy responses.

#### **Docs and support**

##### **Issue: user install guide**

Acceptance criteria:

- GitHub release, install script, deb/rpm, Homebrew documented.

### Issue: operator guide

Acceptance criteria:

- `edge.yml`, modules, updates, backups, logs, diagnostics documented.

### Issue: support runbook

Acceptance criteria:

- Standard triage scripts and escalation checklist documented.

## Recommended Codex issue template

### ## Context

Why this work matters, what part of the Bare Systems edge CLI it supports, and any related architectural constraints.

### ## Goal

The concrete outcome this issue should achieve.

### ## Requirements

- Functional requirement
- Functional requirement
- Non-functional requirement
- Security/compatibility requirement

### ## Implementation notes

- Suggested files/modules
- Known dependencies
- APIs or commands to integrate with
- Any migration or backward-compatibility concerns

### ## Acceptance criteria

- [ ] Behavioral outcome
- [ ] Validation/tests added
- [ ] JSON output or UX requirement satisfied
- [ ] Docs updated if relevant

### ## Out of scope

State explicit non-goals to prevent scope creep.

### ## References

Link to epic, design section, or supporting docs.

## Testing strategy

The testing pyramid should be deliberately stronger than a typical CLI project because this tool is customer-facing and touches privileged runtime operations.



Recommended layers:

**1. Unit tests**

- 2. config parsing
- 3. schema validation
- 4. profile resolution
- 5. output formatting
- 6. redaction
- 7. Portal request/response mapping

**8. Integration tests**

- 9. local Docker Engine with disposable Compose project
- 10. mock Portal
- 11. generated Compose validation
- 12. health/doctor paths
- 13. logs and bundle generation

**14. End-to-end tests**

- 15. install from release artifact
- 16. initialize host directory
- 17. enroll against mock Portal
- 18. start services
- 19. verify health
- 20. update and rollback

**21. Smoke tests**

- 22. package install/uninstall
- 23. `version`, `doctor`, `validate`
- 24. Homebrew install
- 25. `.deb` / `.rpm` install

**26. Upgrade/rollback tests**

- 27. N-1 to N

- 28. channel promotion path
- 29. digest-lock to digest-lock
- 30. failed update rollback

## Recommended test matrix

| Test type                  | Linux amd64 | Linux arm64    | macOS amd64     | macOS arm64     |
|----------------------------|-------------|----------------|-----------------|-----------------|
| Unit                       | Yes         | optional cross | Yes             | Yes             |
| Config/render integration  | Yes         | Yes            | Yes             | Yes             |
| Docker/Compose integration | Yes         | Yes            | no full runtime | no full runtime |
| Package smoke              | Yes         | Yes            | Homebrew        | Homebrew        |
| Upgrade/rollback e2e       | Yes         | Yes            | no              | no              |

Because the real deployment target is a Linux + Docker + systemd host, Linux ARM64 and AMD64 deserve the deepest testing. macOS should focus on operator binary/install usability rather than pretending to be a first-class edge runtime target.

## Portal monitoring and telemetry contract

The exact Portal endpoints are unspecified, so the CLI should target an interface like:

- POST /enroll
- POST /reports/heartbeat
- POST /reports/diagnostics
- optional GET /policy or GET /deployment-config

Recommended heartbeat payload fields:

```
{
  "deviceId": "unspecified",
  "customerId": "unspecified",
  "siteId": "unspecified",
  "cliVersion": "1.2.3",
  "configRevision": "sha256:...",
  "channel": "stable",
  "modules": ["core", "ai"],
  "profiles": ["core", "ai"],
  "containers": [
    {"service": "tardigrade", "state": "running", "health": "healthy"},
    {"service": "bearclaw-web", "state": "running", "health": "healthy"}
  ],
  "health": {
    "summary": "healthy",
    "checks": [...]
  }
}
```

```

},
"stats": {
  "cpu": "...",
  "memory": "...",
  "diskFreeBytes": 0
},
"lastError": null,
"timestamp": "2026-06-13T12:34:56Z"
}

```

Contract recommendations:

- monotonic heartbeat interval with jitter
- local spool for offline periods
- capped retries with backoff
- `lastError` field for support visibility
- separate opt-in for higher-volume metrics vs minimal status

## Documentation set

A production rollout needs four doc tracks:

| Doc type       | Audience                | Required deliverables                                               |
|----------------|-------------------------|---------------------------------------------------------------------|
| User docs      | customers/operators     | install, enroll, start, update, uninstall                           |
| Admin docs     | field/support engineers | config reference, module reference, systemd, networking, backup     |
| Developer docs | Bare Systems engineers  | architecture, schemas, release workflow, testing, Portal mock usage |
| Support docs   | support/on-call         | doctor interpretation, bundle handling, rollback runbook            |

Recommended top-level docs:

- `README.md`
- `docs/install/linux.md`
- `docs/install/homebrew.md`
- `docs/config/edge-yml.md`
- `docs/modules.md`
- `docs/commands.md`
- `docs/portal-contract.md`
- `docs/releasing.md`
- `docs/troubleshooting.md`
- `docs/support-runbook.md`

## Rollout plan

Use staged channels:

- **canary**: internal devices, fastest image/tag movement
- **beta**: friendly customers / pilot environments
- **stable**: default GA channel

Compose digest locks should be generated per release and optionally per channel, so a stable node can be upgraded predictably and rolled back to a known-good prior lock set. Docker Compose supports digest lock/resolve operations that make this model practical. <sup>20</sup>

## Milestone roadmap

| Milestone            | Contents                                                                | Effort |
|----------------------|-------------------------------------------------------------------------|--------|
| Foundation           | CLI skeleton, config loader, JSON output, basic docs                    | Small  |
| Config and rendering | <code>edge.yml</code> , module manifests, compose rendering, validation | Medium |
| Runtime and service  | Docker prereqs, lifecycle commands, systemd install                     | Medium |
| Enrollment/reporting | Portal client, secure enroll, heartbeat                                 | Medium |
| Health/support       | doctor, bundles, health engine                                          | Medium |
| Packaging/release    | release assets, checksums, attestations, deb/rpm/homebrew               | Large  |
| Hardening/GA         | upgrade/rollback, support runbook, beta/stable promotion                | Large  |

## Timeline flowchart



## Rollback procedures

Rollback must be possible at three levels:

1. **config rollback**
2. restore prior `edge.yml`
3. rerender Compose
4. restart services
5. **image rollback**

6. restore prior digest lock file
7. `docker compose up -d` with locked images
8. **package rollback**
9. reinstall previous CLI package/binary
10. verify compatibility with existing config schema
11. if schema migration occurred, require explicit downgrade support

Support runbook minimum:

- collect `bare-systems status --json`
- run `bare-systems doctor --json`
- generate `bare-systems bundle`
- inspect recent release change
- attempt `bare-systems rollback`
- if runtime still degraded, hold on prior stable lock set and escalate

## Open questions and limitations

A few important items remain unspecified and should be resolved before implementation freezes:

| Open question                                                                         | Why it matters                                                  |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Exact Portal API endpoints and auth model                                             | Needed for enroll/report client and retry semantics             |
| Exact container image names, registries, and tagging policy                           | Needed for lock generation and release policy                   |
| Whether rootless Docker must be supported                                             | Affects privilege model, systemd integration, support matrix    |
| Whether Windows is only an operator platform or a supported runtime                   | Determines Winget priority and CLI behavior on Windows          |
| Whether module manifests are shipped inside the CLI, in the repo, or fetched remotely | Affects update policy and offline behavior                      |
| Whether customer environments are air-gapped                                          | Affects install script, package repo design, image preload flow |
| Required telemetry fields and privacy policy                                          | Determines report payload and redaction behavior                |

Where this report recommends specifics, they are design recommendations derived from the documented behavior of Docker Compose, GitHub Actions, Homebrew, Debian APT, RPM, Snap, Windows Package Manager, Sigstore, and Go—not statements about already-existing Bare Systems implementation details.

---

1 Docker Compose

[https://docs.docker.com/compose/?utm\\_source=chatgpt.com](https://docs.docker.com/compose/?utm_source=chatgpt.com)

2 13 15 16 17 18 19 Define services in Docker Compose | Docker Docs

<https://docs.docker.com/reference/compose-file/services/>

3 27 Homebrew Documentation: Taps (Third-Party Repositories)

<https://docs.brew.sh/Taps>

4 14 Docker Engine security

[https://docs.docker.com/engine/security/?utm\\_source=chatgpt.com](https://docs.docker.com/engine/security/?utm_source=chatgpt.com)

5 23 Running variations of jobs in a workflow - GitHub Docs

<https://docs.github.com/actions/writing-workflows/choosing-what-your-workflow-does/running-variations-of-jobs-in-a-workflow>

6 12 docker compose ps | Docker Docs

<https://docs.docker.com/reference/cli/docker/compose/ps/>

7 Merge Compose files | Docker Docs

<https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/>

8 systemd.service

[https://www.freedesktop.org/software/systemd/man/systemd.service.html?utm\\_source=chatgpt.com](https://www.freedesktop.org/software/systemd/man/systemd.service.html?utm_source=chatgpt.com)

9 Set, use, and manage variables in a Compose file with ...

[https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/?utm\\_source=chatgpt.com](https://docs.docker.com/compose/how-tos/environment-variables/variable-interpolation/?utm_source=chatgpt.com)

10 Fragments | Docker Docs

<https://docs.docker.com/reference/compose-file/fragments/>

11 Use include to modularize Compose files | Docker Docs

<https://docs.docker.com/reference/compose-file/include/>

20 docker compose config | Docker Docs

<https://docs.docker.com/reference/cli/docker/compose/config/>

21 Artifact attestations - GitHub Docs

<https://docs.github.com/en/actions/concepts/security/artifact-attestations>

22 Signing Blobs

[https://docs.sigstore.dev/cosign/signing/signing\\_with\\_blobs/?utm\\_source=chatgpt.com](https://docs.sigstore.dev/cosign/signing/signing_with_blobs/?utm_source=chatgpt.com)

24 25 GitHub-hosted runners reference - GitHub Docs

<https://docs.github.com/en/actions/reference/runners/github-hosted-runners>

26 Go 1.13 Release Notes - The Go Programming Language

<https://go.dev/doc/go1.13>

28 apt-secure(8) — apt — Debian trixie — Debian Manpages

<https://manpages.debian.org/trixie/apt/apt-secure.8.en.html>

29 **rpmsign(1)**

[https://rpm.org/docs/6.0.x/man/rpmsign.1?utm\\_source=chatgpt.com](https://rpm.org/docs/6.0.x/man/rpmsign.1?utm_source=chatgpt.com)

30 **Create your package manifest | Microsoft Learn**

<https://learn.microsoft.com/en-us/windows/package-manager/package/manifest>

31 **Classic confinement - Snap documentation**

<https://snapcraft.io/docs/explanation/security/classic-confinement/>

32 **apt-key(8) — apt — Debian unstable — Debian Manpages**

<https://manpages.debian.org/unstable/apt/apt-key.8.en.html>

33 **rpm.org - Spec file format**

<https://rpm.org/docs/latest/manual/spec.html>

34 **Homebrew Documentation: How to Create and Maintain a Tap**

<https://docs.brew.sh/How-to-Create-and-Maintain-a-Tap>